RUBIXDB

Architecture, Engineering & Technical Reference

Manik

First source-grounded edition · Revision 5e8ab125dabf1b77b82f687b88aaf194cf179e84 · 2026-10-20

Contents

Part I — Product and design foundations

Part II — Installation and first use

Part III — SQL language and data model

Part IV — Query processing and execution

Part V — Storage engine and data structures

Part VI — Transactions, concurrency, and reliability

Part VII — Bulk data loading

Part VIII — Server, CLI, and application interfaces

Part IX — Security and operational design

Part X — Performance engineering

Part XI — Testing, debugging, and maintenance

Part XII — Internals and contributor guide

Part XIII — Roadmap and technical limitations

Architecture, Engineering & Technical Reference

Manik

First source-grounded edition Software revision covered: 5e8ab125dabf1b77b82f687b88aaf194cf179e84 Observed publication date: 2026-10-20

Copyright and software-license note

Copyright © 2026 Manik for this book manuscript and its original editorial presentation.

This book is documentation, not a grant of rights to the RubixDB software. The repository revision covered by this edition did not contain a root LICENSE, COPYING, or NOTICE file in the inspected root inventory. Package metadata for the Python client declares Proprietary, while the Node package declares UNLICENSED. Readers and redistributors must therefore consult the repository owner before treating the software as open source or redistributing either the software or this book. No publisher, ISBN, trademark registration, institutional affiliation, or external legal status is asserted here.

The diagrams in docs/book/diagrams/ and the manuscript are original documentation artifacts created for this edition. Their relationship to the software is descriptive; they do not expand the software license.

Edition scope

This book describes the implementation at Git revision 5e8ab125dabf1b77b82f687b88aaf194cf179e84, branch master, as observed on 2026-10-20. The working tree contained unrelated uncommitted changes during investigation. Those changes were not treated as evidence for this edition, and the database implementation was not modified to make the book easier to write.

The evidence order is:

• Current source code and executable tests.

• Current package and build configuration.

• Existing implementation documentation and release notes.

• Architecture specifications and design documents, clearly identified as intent when runtime evidence is absent.

• Analysis, which is labeled as analysis rather than presented as an implementation fact.

The status words used throughout the book are deliberate:

• **Implemented** means the behavior exists in the inspected source and has a relevant test or exercised path.

• **Implemented with limits** means it exists but has explicit bounds or a narrower contract.

• **Partial** means a defined stage or feature slice exists while another stage is deferred.

• **Design intent** means the repository specifies the behavior but the current build does not establish it.

• **Unsupported / not certified** means the project explicitly does not provide or certify the behavior.

• **Unverified** means this edition makes no claim because available evidence was insufficient.

Contents

• Preface

• How to use this book

• Implementation-status note

• Part I — Product and design foundations

- 1. What RubixDB is - 2. Boundaries and architecture - 3. Engineering principles

• Part II — Installation and first use

- 4. Workspace, build, and artifacts - 5. Starting an instance - 6. First query and lifecycle

• Part III — SQL language and data model

- 7. Relational values and schema - 8. SQL surface - 9. Parameters, constraints, and errors

• Part IV — Query processing and execution

- 10. From request to result - 11. Binding and authorization - 12. Planning and physical access - 13. Execution operators

• Part V — Storage engine and data structures

- 14. The durability boundary - 15. The WAL - 16. Memtables and SSTables - 17. Catalogs, rows, and indexes - 18. Recovery and integrity

• Part VI — Transactions, concurrency, and reliability

- 19. Transaction model - 20. Concurrency limits and conflicts - 21. Cancellation, shutdown, and failure

• Part VII — Bulk data loading

- 22. COPY and LOAD DATA - 23. File safety and batch semantics

• Part VIII — Server, CLI, and application interfaces

- 24. The HTTP service - 25. The CLI and product binary - 26. Frontend and clients - 27. Framework integrations

• Part IX — Security and operational design

- 28. Authentication and authorization - 29. Secrets, network, and filesystem boundaries - 30. Observability without data leakage

• Part X — Performance engineering

- 31. Performance model - 32. Measurement discipline

• Part XI — Testing, debugging, and maintenance

- 33. Test architecture - 34. Debugging workflow - 35. Release and compatibility

• Part XII — Internals and contributor guide

- 36. Repository tour - 37. Making a change

• Part XIII — Roadmap and technical limitations

- 38. What is present - 39. What is deliberately absent

• Appendices

- A. SQL reference map - B. CLI reference - C. Configuration reference - D. HTTP route reference - E. Error and limit reference - F. Type and compatibility matrix - G. Source-code map - H. Troubleshooting - I. Glossary and acronyms - J. Implementation inventory

Preface

RubixDB is best understood by following its boundaries. A command typed into rubixdb.exe is not necessarily executed by an embedded SQL engine inside the command-line client. A SQL request arriving through HTTP does not bypass the relational implementation. A table row is not merely a JSON object stored behind a web route. The product is a stack: a client and host, an authenticated HTTP service, a SQL front end, relational row and index primitives, and a Rust storage engine with explicit durability and recovery boundaries.

The repository also contains a large body of design history. That history is valuable, but it creates a documentation hazard: a specification can describe an intended phase while a source module intentionally implements only a slice. This book therefore

makes status and evidence part of the prose. A reader should be able to move from a claim to a module, a symbol, a test family, or a named design document without relying on an invented line number.

The goal is not to make RubixDB appear larger than it is. The goal is to make its actual engineering legible: what is implemented, what is bounded, what is conservative by design, and what remains outside the current product contract.

How to use this book

Start with Part I if you are evaluating the architecture. Use Part II for a local build and first query. Parts III–VII explain the database itself. Part VIII is the application-integration reference. Parts IX–XI are for operators and maintainers. Part XII is a source tour for contributors. The appendices are lookup material.

The repository-relative paths in this book are stable navigation points. For example, sql/src/plan/mod.rs::build_plan identifies the planner entry point, while api/src/routes/mod.rs::build_router identifies the HTTP route assembly. The book intentionally avoids citing volatile line numbers.

The companion files are:

• TECHNICAL_INVENTORY.md — the evidence register used while writing.

• BOOK_SPEC.md — publication structure and conventions.

• diagrams/*.mmd — editable Mermaid-compatible diagrams.

• PUBLICATION_README.md — build and update instructions.

Implementation-status note

This edition covers a single-node, local-first product shape. The normal product executable is rubixdb.exe, which can host the API in-process, manage a local instance, serve the embedded frontend when configured, and act as an HTTP SQL client. rubixdb-api.exe is also emitted by the workspace, but the repository records the standalone deployment shape as not certified for the current v1 product path. The book does not treat it as equivalent to the integrated product without qualification.

The current server exposes bearer API-key authentication and reader/admin roles. It does not expose a complete key-management lifecycle with create, reveal, rotate, revoke, or rotation-grace routes. The frontend records those limits honestly. The book does the same.

The repository contains SQL, storage, client, adapter, and frontend tests. Not every platform, package-publishing workflow, or GitHub workflow was executed during this edition. Such items are labeled unverified.

Part I — Product and design foundations

1. What RubixDB is

RubixDB is a Rust database product with a local instance manager, an HTTP API, a relational SQL layer, a key-value surface, and an LSM-oriented persistent storage foundation. Its implementation is organized as a Cargo workspace:

```
rubixdb          core storage, catalog, relational primitives, WAL, LSM
rubixdb-sql      parser, binder, planner, executor, SQL types and limits
rubixdb-api      Axum HTTP service and route/auth/observability layers
rubixdb-cli      product binary, CLI, host, GUI/static frontend integration
rubixdb-instance instance lifecycle and filesystem/port/credential ownership
```

The root package description calls the product an embedded, multi-engine database, but the current inspected implementation should be described more precisely: the repository contains a mature LSM and relational path plus a substantial SQL/API surface, while several architecture documents describe future or phase-bound components. This book uses the code and tests to determine present behavior rather than treating every planned engine as available.

The product is suited to local or single-node deployments where an application can reach an HTTP endpoint on the same machine or a deliberately configured allowed origin. The clients and framework adapters are designed for backend

applications, while the React frontend supplies a browser console. The HTTP service is not a PostgreSQL wire-protocol server, and it is not documented here as a drop-in replacement for PostgreSQL or MySQL drivers.

Figure 1. Product context

Source: diagrams/01-product-context.mmd.

The important boundary is the HTTP service. The CLI, frontend, Python client, Node client, FastAPI adapter, Express adapter, and Django helper are clients of that service. The core engine is reached behind the API and SQL layers. This separation permits typed errors, authentication, resource limits, and observability to be enforced at one service boundary.

2. Boundaries and architecture

The dependency direction is intentional. rubixdb-sql depends on the core rubixdb crate because binding and execution need catalog, table, transaction, and storage primitives. The core does not depend on the SQL crate. This keeps the storage engine usable below the SQL layer and prevents SQL concerns from becoming a hidden dependency of the durability foundation.

The API crate depends on both. It owns request authentication, route selection, HTTP error mapping, request limits, service metrics, and the translation from API principals to SQL authorization contexts. The CLI has two roles documented in cli/src/main.rs: client role and host role. In client role it sends SQL over HTTP; in host role it owns local instance acquisition and starts the server in process. That is why one rubixdb.exe can be both the interactive command and the local database host.

The instance crate is the lifecycle boundary. It creates or discovers a named instance directory, acquires an operating-system lock, binds or reuses a port, persists a manifest, loads or generates credentials, and verifies an already-running process with an HTTP handshake. The source explicitly avoids trusting a PID alone. A lock with no verifiable identity is reported as locked-but-unverifiable rather than automatically broken.

3. Engineering principles

Several principles recur across the code:

• **Make boundaries explicit.** The SQL planner does not execute plans; the executor does not silently ignore unsupported plan variants; the API does not invent an independent table-authorization system.

• **Prefer fail-closed behavior.** WAL corruption stops trust at the first questionable segment. SSTable discovery treats validation failure as an error rather than silently excluding a table. Authentication rejects missing or unknown credentials.

• **Bound resource use.** Request bodies, result sizes, open SQL sessions, statement deadlines, range limits, import rows, import bytes, and observability registries are bounded.

• **Keep sensitive values out of diagnostics.** API-key debug output is redacted, security events use route patterns rather than raw paths, and observability avoids storing SQL text and credentials.

• **Use measurement for performance decisions.** The repository contains benchmarks and performance documents, but this book publishes only figures with identifiable workload and environment.

• **Distinguish durable truth from derived read structures.** In the current phase boundary, the WAL remains the durable source while SSTables provide a derived read path; an SSTable is not presented as the sole durable copy.

Part II — Installation and first use

4. Workspace, build, and artifacts

The verified toolchain during this edition was:

```
rustc 1.98.1 (48a229cea 2026-09-01)
cargo 1.98.1 (797e8a9bc 2026-08-05)
Node v24.21.0
npm 11.19.0
Python 3.14.7
```

The workspace uses Rust edition 2021. The root Cargo.toml lists the workspace members ., api, sql, cli, and instance. The root package is version 0.1.0 and is marked publish = false. Client packages carry their own version and release metadata.

Build the integrated release product with:

```
cargo build --release -p rubixdb-cli
```

The principal output is:

```
target\release\rubixdb.exe
```

The release directory also contains Cargo intermediates, dependency artifacts, debug symbols, and the separately built rubixdb-api.exe. Those files explain the size and density of target\release; they are not all deployment inputs. The integrated product path is the one to use for ordinary local operation.

The API crate is also compiled because it is a workspace member and because the CLI host uses its library code. A separately emitted rubixdb-api.exe requires standalone configuration such as RUBIXDB_DATA_DIR, but the repository's recorded product status does not certify that shape as the normal v1 deployment. Do not start both binaries for one instance.

5. Starting an instance

The CLI help in cli/src/main.rs is the authoritative command surface for the product binary:

```
rubixdb gui                 find/start the local instance and open the console
rubixdb cli                 explicit interactive SQL client alias
rubixdb menu                launcher behavior for a double-click
rubixdb instance list       list known local instances
rubixdb instance status     inspect one instance
rubixdb status|check|backup|restore|storage|maintenance
rubixdb                     interactive SQL client
rubixdb -c "SQL"            execute a statement and exit
rubixdb -f script.sql       execute a script file and exit
```

A verified release binary invocation was:

```
.\target\release\rubixdb.exe -c "SELECT 1"
```

It returned one row with value 1. The command also exercises local instance discovery/startup when no explicit remote URL is configured.

To start the interactive product:

```
.\target\release\rubixdb.exe
```

The default local API port is normally 302, although the instance manifest may persist a different available port. Use rubixdb instance list rather than assuming a port after relocation or collision recovery.

6. First query and lifecycle

The first-touch lifecycle is:

• Resolve the instance name and directory.

• Attempt the operating-system lock.

• If the lock is owned, bind the persisted or default port, load/persist the manifest, and load/generate credentials.

• Open storage and perform recovery.

• Start the embedded Axum service and optional frontend distribution.

• If another process owns the lock, perform a real identity handshake and attach only when the response verifies the expected instance.

Figure 2. Startup lifecycle

Source: diagrams/02-startup-lifecycle.mmd.

The CLI can connect to an explicit HTTP service through environment variables:

```
$env:RUBIXDB_API_URL = "http://127.0.0.1:302"
$env:RUBIXDB_API_KEY = "<key supplied through a secret mechanism>"
```

```
.\target\release\rubixdb.exe -c "SELECT 1"
```

The client source deliberately does not accept an API-key command-line option. That avoids shell history and process-list exposure. With an explicit URL and an interactive terminal, the CLI can prompt without echoing the key. With local auto-discovery, the instance credential is loaded from the instance directory rather than typed.

Shutdown is bounded. The service stops accepting work, drains what it can within the configured budget, and closes storage and pools. The exact operator subcommands should be read from rubixdb --help for the current build; the book does not invent flags not present in that output.

Part III — SQL language and data model

7. Relational values and schema

The relational foundation lives in src/relational. It provides RelationalType, RelationalValue, row representation, key encoding, TableStore, indexes, transaction primitives, and foreign-key support. A table row is stored through the catalog and row-storage layers; it is not a JSON document stored directly by the HTTP handler.

The current type machinery includes integer families, real/double floating values, exact decimals, text, blobs, booleans, date, time, timestamp, and additional JSON/variant-related surfaces represented in sql/src and the phase-specific tests. Exact type behavior matters. A decimal value is not silently rescaled merely because the target column has a different scale. Client libraries therefore provide explicit parameter wrappers for values whose inferred type would otherwise be ambiguous.

The catalog owns object identity and schema metadata. The SQL binder resolves table, column, index, database, and schema names against that catalog. The relational row layer uses ordinals and encoded keys; this is one reason apparently simple DDL changes such as dropping a column can have broad consequences and are recorded as a separate design problem rather than guessed into the product.

8. SQL surface

The parser and test tree show a broad, evolving surface. Verified test families include:

• statements and expressions in sql/src/parse, sql/src/ast, and scalar/cast/datetime tests;

• DDL and DML in phase-specific modules;

• joins, outer joins, hash/merge access, subqueries, correlated queries, CTEs, set operations, and recursive queries;

• aggregation, windows, grouping sets, ROLLUP, CUBE, and related functions;

• indexes, primary-key access, foreign keys, conflict handling, and RETURNING;

• temporal, regular-expression, JSON/variant, and additional scalar functions;

• COPY and LOAD DATA in sql/src/copy and related tests.

This list is a map to implementation evidence, not a claim that RubixDB is syntactically or behaviorally identical to PostgreSQL. The compatibility document docs/SNOWFLAKE_SQL_COMPATIBILITY.md covers particular compatibility work, but external dialect compatibility must be checked feature by feature.

The smallest verified SQL example is:

```
SELECT 1;
```

A parameterized example used by both client READMEs is:

```
SELECT id, name FROM people WHERE id = $1;
```

Parameters are typed values, not interpolated text. The API request represents SQL and an optional typed params array. The frontend wire type is defined in frontend/src/api/types.ts::SqlRequestBody and SqlValue.

The following categories should not be assumed available merely because a familiar database supports them:

Feature	Current treatment
PostgreSQL wire protocol	Unsupported / not claimed
MySQL wire protocol	Unsupported / not claimed
Django ORM over HTTP	Unsupported; use raw SQL through the Python client
Distributed replication/sharding	Unverified and not part of the current single-node product contract
Arbitrary server-side file export	Not claimed; current bulk-load feature is an import path
Complete API-key lifecycle	Not implemented in the current API routes

9. Parameters, constraints, and errors

The binder is the main correctness boundary for identifiers, types, privileges, and statement shape. It produces a BoundStatement. The SQL API then carries this object into planning and execution rather than re-resolving names in a later layer.

The Python client infers or accepts explicit values such as Integer, Bigint, Real, and Decimal. The Node client provides corresponding param helpers and preserves BIGINT as JavaScript bigint in results. These behaviors are documented in clients/python/README.md and clients/node/README.md and exercised by their type/contract tests.

Important error classes include parse and binding errors, authorization failures, type/conversion errors, constraint and duplicate-key conflicts, transaction conflicts, rate/session limits, resource limits, timeouts, connection failures, storage failures, and server errors. The HTTP layer maps them to structured responses. Client libraries map them to typed exceptions rather than leaking raw httpx or undici exceptions.

A 409 is not one universal retry signal. A lost write race is retryable only when the application can safely rerun the entire transaction. A duplicate-key or integrity conflict generally fails again if blindly repeated. The clients document and test this distinction.

Part IV — Query processing and execution

10. From request to result

The SQL request path is represented by the following stages:

```
SQL text + typed parameters
■■|
■■v
Parser / AST
■■|
■■v
Binder + catalog + authorization
■■|
■■v
BoundStatement
■■|
■■v
Planner + validation + physical access choice
■■|
■■v
Execution operators
■■|
■■v
Transaction / TableStore / IndexBuilder
■■|
■■v
Typed result, write result, DDL result, or typed error
```

Figure 3. SQL request lifecycle

Source: diagrams/03-sql-lifecycle.mmd.

The HTTP route in api/src/routes/sql.rs is an adapter. It parses the JSON envelope, creates or retrieves an SQL session when an explicit transaction is involved, invokes the SQL layer, and serializes a result or error. The route does not contain a second SQL parser or an independent execution engine.

11. Binding and authorization

sql/src/bind resolves names and builds bound expressions and statements. The bound representation retains catalog identity and typed information. This is important for correctness: the planner can relocate or inspect bound objects without performing a second, potentially inconsistent name lookup.

Authorization has two layers with a deliberate division of responsibility. The API authenticates a bearer key and maps it to a Principal with reader or admin role. api/src/auth.rs::to_sql_auth_context is the one explicit translation into the SQL authorization context. A SQL POST endpoint must accept both read and write statement kinds, so the API route cannot decide the final privilege from the HTTP method alone. The SQL binder performs statement-level authorization.

This avoids a common failure mode in layered systems: one layer silently grants or denies a different set of objects than another. The API decides who is calling and the broad route role; the SQL layer decides whether the specific statement is authorized.

12. Planning and physical access

sql/src/plan documents its own boundary: it turns a bound statement into a logical/physical plan and does not execute it. Plan variants cover query plans, write/DDL pass-through forms, transaction control, COPY, and explain structures. The code comments are explicit that not every variant is a physical execution node.

For SELECT, the planner can build a physical plan and choose a PhysicalAccess decision. The same access decision is reused for the target row set of UPDATE and DELETE, reducing the risk that reads and writes silently use divergent predicate behavior. Access selection can consider primary-key/range and index structures where the current planner and executor support them.

The planner validates the resulting plan before returning it. Planning metrics record successful and failed plan construction. Cost and relation-statistics modules exist, but a cost model should not be described as a universal optimizer or a guarantee of a particular index choice without a test or explain output demonstrating that choice.

13. Execution operators

The executor in sql/src/exec consumes plans and produces typed rows or typed errors over the real TableStore, IndexBuilder, and Transaction primitives. Its operator modules include access, expressions, joins, outer joins, aggregates, windows, subqueries, recursive processing, set operations, scalar functions, and write support. The exact statement support is constrained by the executor and phase tests.

The row context associates a table reference with a row and carries aggregate, window, grouping-key, and grouping-mask state. This allows expressions to distinguish ordinary row values, aggregate results, window results, and grouping-set nulls. A null-extended outer-join side is represented in a way that causes missing ordinal reads to resolve as SQL NULL.

Execution is bounded by budgets and cancellation. SQL limits are surfaced as RESOURCE_LIMIT-style errors rather than allowing an unbounded operation to consume the process. A statement deadline is propagated from API configuration into the executor, making the service’s timeout setting the operator-facing source of truth.

Part V — Storage engine and data structures

14. The durability boundary

The storage path is intentionally layered:

```
logical mutation
   -> transaction/table overlay
   -> write batch coordinator
   -> WAL append and durability wait
```

```
-> mutable memtable
-> immutable memtable
-> derived SSTable flush
```

Figure 4. Storage write path

Source: diagrams/04-storage-write-path.mmd.

The key guarantee in the current LSM comments is that a memtable mutation is applied only after the WAL durability completion says the operation is durable. SSTables add a read source derived from durable data; they are not the sole durable copy and the current phase does not allow the read structure to purge the WAL merely because an SSTable exists.

This is a conservative boundary. It costs space and can leave cleanup work for later phases, but it keeps recovery authority clear. A reader should not infer from the presence of a .sst file that the WAL can always be deleted.

15. The WAL

The WAL implementation in src/wal follows a framed on-disk format. The segment header contains the RBXWALv1 magic, version, segment ID, and reserved flags. The default segment header length is 24 bytes; a frame contains a length and CRC32C followed by a record body. The record body carries a sequence number and operation tag. PUT, DELETE, checkpoint marker, and grouped operations are represented by explicit tags in src/wal/format.rs and src/wal/ops.rs.

FileWal is intentionally single-writer. Its mutable bookkeeping is not Sync, and a directory-level exclusive lock prevents a second writer process from opening the same WAL. Inspectors use compatible read behavior where appropriate. This is a type-system and operating-system invariant rather than merely a comment.

Recovery distinguishes a permitted torn tail from corruption and validates lengths before allocating or indexing. The current implementation deliberately amends the original design rule to stop trusting records after the first corrupted segment. This is fail-closed: a later segment is not treated as authoritative once the sequence position before it is questionable.

Immediate durability waits for the append boundary. Group commit coalesces concurrent sync calls and publishes a durability watermark only after the grouped fsync succeeds. A FileWal configured for group commit but used without the GroupCommitter remains safe, although it loses batching efficiency and behaves like a stricter immediate path.

16. Memtables and SSTables

Memtables provide mutable and immutable in-memory sources. The active memtable is protected for concurrent access; immutable memtables are drained by flush work. Backpressure limits the number of pending immutable structures so a sustained write rate cannot grow memory without a bound.

RUBIC SSTables are immutable sorted files. The current format includes data blocks, a bloom filter block, a sparse index block, and a fixed footer. Publication uses a temporary file and rename discipline. At startup, src/sstable::discover removes .sst.tmp files as interrupted builds, validates published tables, and fails rather than silently excluding a corrupt table. Live table discovery is based on file naming and validation in the current phase boundary, not an assumed complete manifest protocol.

The read source preserves tombstones until the LSM merge can apply visibility rules across sources. A per-source reader must not collapse a tombstone too early or an older value could incorrectly reappear from another source.

Figure 5. Read path

Source: diagrams/05-read-path.mmd.

17. Catalogs, rows, and indexes

The catalog provides database, schema, table, column, index, and privilege metadata. The relational row layer applies typed values to encoded keys and uses the LSM engine’s ordinary get/range primitives. TableStore owns row-level access while transaction overlays track changes before commit.

Indexes are not a separate universe from rows. Their builders and lookup paths integrate with the relational transaction and planner layers. The planner’s access decision is reused by DML target selection where possible. Foreign-key tests and phase-specific DDL/DML tests provide evidence for the implemented constraint surface.

Schema evolution is deliberately conservative. Column ordinals participate in row representation and key/index relationships, so a destructive operation such as dropping a column is not documented as available merely because a parser token could be added. The repository records that class of change as a separate architectural task.

18. Recovery and integrity

Recovery ties together WAL replay, memtable reconstruction, SSTable discovery, manifest/catalog state, and integrity checks. It is cross-cutting rather than a standalone recovery directory. The engine opens storage, validates on-disk structures, and replays durable operations according to the implemented recovery rules.

Integrity behavior is fail-closed where ambiguity could cause silent wrong results. A malformed header, invalid CRC, impossible length, mismatched segment ID, unsupported version, or invalid SSTable footer is an error. A torn final write has a narrower recovery treatment described in the WAL specification and tests; it must not be generalized to arbitrary corruption.

The repository contains extensive recovery and fault-injection documentation. This book distinguishes those certified tests from unverified environmental claims such as power-loss behavior on every filesystem and operating system.

Part VI — Transactions, concurrency, and reliability

19. Transaction model

The SQL session registry exists for explicit transaction sessions. A request that executes a standalone statement can complete without holding an open SQL session. BEGIN, subsequent statements, and COMMIT/ROLLBACK use a session identifier carried in the HTTP SQL envelope. Session lifetime and idle time are bounded by API configuration.

The relational layer exposes transaction state, read views, overlays, savepoints, row changes, and transaction metrics. A write is staged through the transaction and reaches the storage engine through a durable batch boundary. A successful commit makes the transaction’s changes visible according to the implemented snapshot/conflict model; a failed commit returns a typed conflict or constraint error.

The client transaction helpers are intentionally structured around commit-before-success. In the framework adapters, a response is not treated as successful until the database transaction has committed. That prevents an application from returning HTTP 200 and only afterward discovering a lost write race.

20. Concurrency limits and conflicts

The server and clients surface a per-principal open-session limit. The published client documentation describes 50 open transactions per key as the current limit. Exceeding it is a rate/session error rather than an indefinite wait. Framework examples include a test for this behavior.

The storage and transaction design uses snapshot-style conflict behavior with a first-committer-wins shape for the documented write race. A RubixConflictError or equivalent HTTP 409 means the complete transaction may be retried if it is safe to rerun. A duplicate key, foreign-key, check, or other integrity conflict also uses a conflict-class status in parts of the API, but rerunning the same operation is generally not correct.

Concurrency is bounded in more than one place: WAL writer ownership, transaction/session count, immutable memtables, SQL statement work, result rows, request body size, rate limiting, and observability storage. These limits are part of reliability, not merely tuning knobs.

21. Cancellation, shutdown, and failure

The frontend attaches an abort signal to the SQL fetch. The API and executor have cancellation paths so a canceled request is not represented merely as a spinner disappearing from the browser while the server continues without a bound.

The product shutdown path stops accepting new work, drains in-flight work for a bounded time, rolls back or closes abandoned transactions, and closes the pool/storage resources. The Node and Python clients expose graceful close methods. Express and FastAPI examples connect application shutdown to client shutdown.

When a server disappears mid-query, clients classify the failure as a typed connection or timeout error and the circuit breaker can open after repeated failures. Recovery after the service returns is probed through the breaker's half-open path. These client behaviors are tested in client chaos suites; platform-specific failure injection remains subject to the test environment.

Part VII — Bulk data loading

22. COPY and LOAD DATA

Bulk loading is implemented in sql/src/copy and exposed through SQL statements and POST /v1/copy where the route and configuration permit it. The loader supports CSV and JSON readers. It resolves target columns, converts values to the target relational types, and feeds batches to an engine sink.

COPY in this product should not be assumed to mean every feature of PostgreSQL's command. The current implementation is an import surface. The code includes format, delimiter, quote, NULL-string, header, skip-row, input-byte, row-count, row-size, batch-row, batch-byte, and duration limits. Invalid delimiter/quote combinations and invalid option values are refused.

The default loader limits in sql/src/copy/mod.rs include a 1 GiB input bound, 50,000,000 rows, 1 MiB maximum row size, 500 rows per batch, and an 8 MiB maximum batch byte budget. These are implementation defaults, not a universal capacity guarantee; API or statement configuration can affect the effective envelope.

23. File safety and batch semantics

Figure 7. Bulk-load pipeline

Source: diagrams/07-bulk-load.mmd.

The server-side file path is validated through the import-directory boundary. The route must not become an arbitrary filesystem read primitive. The loader reads records incrementally and holds one batch and one record rather than materializing an entire input.

The transaction semantics are intentionally not "all input or nothing." Each batch is atomic. The first failing batch stops the load; batches before that failure remain committed; the error reports the relevant class and progress. Error classes include encoding, malformed CSV/JSON, conversion, row-too-large, limit, batch/engine-limit, conflict, I/O, and cancellation.

This design makes recovery and retry operationally meaningful. A caller can use the reported committed progress and skip-row mechanism where appropriate, but should not assume that blindly rerunning the entire file is idempotent.

Unsupported or unverified claims include cloud object-store stages, COPY TO, error-skipping modes, and a general client-streaming protocol. They are omitted from the supported feature list.

Part VIII — Server, CLI, and application interfaces

24. The HTTP service

The API is an Axum router assembled in api/src/routes/mod.rs. Route middleware records bounded metrics, applies security headers, applies CORS only from an explicit allow-list, authenticates bearer keys, and records selected security events. API responses receive Cache-Control: no-store; static frontend assets can remain cacheable.

The public liveness route is /healthz. Most useful state and data routes require authentication. Readiness and identity endpoints are authenticated in the current route assembly, so an unauthenticated monitoring check should not assume that /readyz is public. Always consult the current route source and deployment configuration.

The main route families are:

Family	Examples	Purpose
Health and identity	/healthz, /readyz, /v1/whoami	liveness, readiness, principal identity
Status	/v1/status, /v1/metadata	instance/storage metadata
SQL	POST /v1/sql	typed SQL request/response and sessions
Bulk load	POST /v1/copy	controlled import path
KV	/v1/kv, /v1/range	key-value and range access
Snapshots	/v1/snapshots	snapshot lifecycle
Catalog	/v1/catalog/*	database/schema/table/index metadata and DDL surfaces
Metrics	/v1/metrics*, compaction routes	quantitative service/storage state
Observability	/v1/observability/*	sessions, queries, events, version
Admin	/v1/admin/*	operator functions such as backup/check/shutdown where configured

The frontend ApiClient mirrors these paths and wire types. It sends Authorization: Bearer <key>, serializes JSON request bodies, maps 401 into session handling, and converts non-2xx responses into ApiRequestError instances.

25. The CLI and product binary

The CLI is a client of the HTTP API in its client role. Its source comments explicitly reject a second embedded SQL engine in the CLI. The host role starts the server in process and can serve the frontend distribution. This arrangement gives a local user a single executable while preserving one SQL/API implementation.

The command surface includes interactive SQL, -c, -f, instance listing/status, GUI/launcher behavior, and operator command families. Credentials are not accepted as command-line arguments. Explicit remote connection uses environment variables; local discovery reads instance credentials from the instance directory.

rubixdb-api.exe is a separate binary target, not a second binary that must be started alongside rubixdb.exe. It requires standalone configuration and is recorded as not certified for the normal product path. The integrated executable is the correct entry point for the examples and frontend.

26. Frontend and clients

Figure 8. Integration flow

Source: diagrams/08-integration-flow.mmd.

The React frontend is a typed browser client. frontend/src/api/client.ts is the single fetch boundary for session-authenticated server calls. frontend/src/api/types.ts mirrors JSON shapes, including tagged SQL values and typed result variants. The SQL console uses POST /v1/sql; catalog queries use the catalog routes; operational pages use status, metrics, and observability routes.

The Data Integration page has four tabs: Import files, Connect from code, API keys, and Connection status. Its import path is a frontend file parsing/load workflow; its connect tab shows runnable examples loaded from the repository examples and documents limits. The API Keys tab does not pretend that unsupported server routes exist. The status tab uses existing read-only endpoints and derived display calculations; the server currently does not provide every desired key-management or per-User-Agent metric.

The Python client is installed from its package project and exposes both sync and asyncio forms:

```
import rubixdb

db = rubixdb.Client()
rows = db.query("SELECT id, name FROM people WHERE id = $1", [1])
print(rows.first().name)

with db.transaction() as tx:
■tx.execute(
■■"UPDATE accounts SET balance = balance - $1 WHERE id = $2",
■■[10, 1],
■)
```

```
db.close()
```

The Node client is ESM/CJS-compatible:

```
import { Client } from "@rubixdb/client";

const db = new Client();
const rows = await db.query(
  "SELECT id, name FROM people WHERE id = $1",
  [1],
);
console.log(rows.first());
await db.close();
```

Both clients use one pooled client per process, typed parameters, transactions, bounded retry policies, circuit breaking, redacted secrets, streaming helpers, bulk inserts, health checks, and graceful shutdown. Neither is a database wire-protocol driver.

27. Framework integrations

FastAPI uses rubixdb.integrations.fastapi. The adapter owns an async client through the application lifespan, supplies client/transaction dependencies, maps typed errors to HTTP statuses, provides NDJSON response helpers, and exposes health/readiness patterns. The runnable application is in examples/fastapi; its README documents pip install -r requirements.txt, environment variables, and uvicorn startup.

Express uses @rubixdb/client/express. attach(db) puts the client on the request, transaction() provides request transaction behavior, errorHandler() maps typed errors, ndjson streams results with pipeline support, and gracefulShutdown drains on process signals. The runnable example is in examples/express.

Django uses rubixdb.integrations.django for raw SQL. It provides a process client, transactional view support, middleware, NDJSON, and health/readiness views. The Django ORM is not supported over this HTTP path because an HTTP SQL client does not supply the DB-API semantics, server-side cursors, savepoints, introspection, and backend contract that the ORM expects. The repository explicitly points that work to a separate wire-protocol roadmap rather than faking an ORM backend.

Part IX — Security and operational design

28. Authentication and authorization

The API accepts bearer credentials in the Authorization header. Configuration parses entries in the form name:role:key, with reader and admin roles. The configuration requires at least one admin key so a deployment does not become permanently unable to perform mutating operations.

The role hierarchy is simple: admin satisfies reader requirements. GET and HEAD routes generally require reader access; mutating routes require admin; /v1/admin/* always requires admin. SQL is a deliberate exception at the HTTP method layer because a POST body may contain a read-only SELECT or a mutation. The SQL binder then applies statement-level authorization.

Authentication failures create bounded counters and security events without storing the presented secret. Authorization failures are distinct from missing/invalid authentication. This distinction is visible in API error classes and operational metrics.

There is no implemented key-creation/reveal/rotate/revoke route in the current server. This means secret rotation is an operational deployment concern outside the current API route set, not a feature the frontend should imply exists.

29. Secrets, network, and filesystem boundaries

API-key debug representations use <redacted>, and configuration debug output reports only a redacted key count. Client libraries keep keys in secret wrappers and exclude them from strings, logs, errors, and traces. Security events use route templates and bounded object identifiers rather than raw URI values.

The default product model is local and loopback-oriented. A separately hosted frontend requires an explicit CORS allow-list; the server does not wildcard origins. TLS-capable client configuration preserves certificate and hostname verification rather than offering an insecure disable switch. Redirects and ambient proxy configuration are deliberately constrained in the clients.

Bulk-load file access is bounded by an import-directory configuration and validated paths. The API sets body limits in relation to configured key/value limits rather than relying only on a framework default. SQL, result, session, and statement limits reduce denial-of-service exposure but are not a substitute for network isolation and process-level controls.

30. Observability without data leakage

Figure 9. Observability boundaries

Source: diagrams/09-observability-boundaries.mmd.

The observability module separates quantitative metrics, diagnostic state, and events. It uses bounded registries and fixed route/label sets. User SQL text, keys, filesystem paths, and arbitrary error strings are not stored as metric labels. The sampler reads counters and operating-system values without mutating database state.

Endpoints include service/storage metrics, system metrics and time series, query/session registries, events, and version. Security events cover authentication failures, admin actions, catalog drops, and selected outcomes. This is an event record of what the current server actually records; it is not the audit trail for a key-management lifecycle that the server does not implement.

Operational readers should treat metrics as bounded diagnostics. A missing observation, unavailable lock probe, or advisory disk forecast is not automatically equivalent to a healthy state. The source distinguishes Held, NotHeld, and Unavailable lock states.

Part X — Performance engineering

31. Performance model

RubixDB performance is the product of several boundaries:

• SQL parsing, binding, authorization, and planning;

• access-path selection and row materialization;

• transaction overlay and conflict behavior;

• WAL append, fsync or group commit, and batching;

• memtable and immutable-memtable pressure;

• SSTable lookup, bloom filtering, sparse indexing, and merge work;

• HTTP serialization, client pools, and framework overhead;

• bulk-load conversion and batch transaction size.

Theoretical complexity is not a measured result. A primary-key lookup may avoid a full scan, but the measured latency still includes the API, authentication, JSON, transaction, and storage path. A group commit can reduce fsync frequency for concurrent writers, but it introduces a batching window and must be evaluated under the target workload.

The repository includes Criterion benchmarks and phase-specific performance documents. The root Cargo.toml identifies WAL, append, write-batch, table-store, secondary-index, and transaction benchmark targets. Some test modules also contain differential and comparison measurements. These should be rerun on the target machine before publishing new capacity claims.

32. Measurement discipline

A reproducible RubixDB benchmark record should include:

• Git revision and build profile.

• Rust, OS, CPU, memory, and storage device.

• Dataset size, key/value distributions, schema, indexes, and cache state.

• Operation mix and concurrency.

• Whether the measurement includes HTTP, client serialization, fsync, and recovery.

• Warm-up, sample count, p50/p95/p99 method, and variance.

• Whether compaction or flush work ran concurrently.

• The exact command or benchmark target.

The client load tests in clients/python/tests/test_load.py and clients/node/test/contract/load.test.ts contain actual workload harnesses. Earlier project evidence measured approximately 9,000 requests/s for the Node client and approximately 910 requests/s for the Python synchronous client on one shared development machine, but those values are not product guarantees and should be treated as historical evidence tied to that run. The Python asyncio test documents an anyio-version sensitivity observed in the environment. This edition does not generalize those figures to other hardware.

Part XI — Testing, debugging, and maintenance

33. Test architecture

Testing is distributed by layer:

• Rust unit tests in core storage, WAL format/recovery, memtables, SSTables, catalog, relational types, indexes, transactions, and compaction.

• SQL parser, binder, authorization, planner, executor, type, function, phase corpus, differential/reference-model, and query-surface tests in sql/src and sql/tests.

• API unit and integration tests in api/src and api/tests, including route/auth/error/observability/security behavior.

• CLI and instance lifecycle tests in cli/tests and related crate test modules.

• Python client tests covering types, error mapping, release checks, contract behavior, load, chaos, and framework examples.

• Node client Vitest unit/contract tests and ESM/CJS build/type checks.

• Frontend Vitest tests, Playwright design/accessibility/integration suites, and generated-snippet checks.

Feature-gated tests are a key status signal. Fault-point and simulated-filesystem tests are not part of every product build. A test under a feature gate should not be described as a default production behavior without checking how the feature is enabled.

34. Debugging workflow

A disciplined investigation follows the same boundaries as the product:

• Reproduce with a minimal SQL or HTTP request.

• Record the Git revision, build profile, environment, instance name, and data directory.

• Determine whether the failure is client, HTTP/auth, SQL parse/bind/plan/execute, transaction, catalog, or storage.

• Use request IDs, typed error codes, query/session observability, and safe logs without exposing keys or SQL parameters.

• Inspect storage with read-only tools where available; do not treat a recovery command as read-only without verifying its implementation.

• Add a regression test at the narrowest layer that reproduces the defect.

• Run the relevant suite plus the full workspace gate before changing the status record.

A 401 indicates missing/invalid bearer credentials or a wrong instance key. A 403 indicates an authenticated principal lacks the required role. A 409 must be classified as write conflict versus integrity conflict. A 429 may represent rate limiting or the open-session limit. A connection refusal may mean the instance is not running, the port in the manifest changed, or a client is pointed at the wrong URL.

35. Release and compatibility

The Rust workspace uses version 0.1.0 in the inspected manifests. The Python client declares Python >=3.9 and uses httpx; the Node client declares Node >=20.18.1 and uses undici 7. The package READMEs define compatibility against the POST /v1/sql API of the 2026-10 builds, including typed parameters and session_id transactions.

Client version gates compare package versions and changelog entries. The repository contains GitHub workflows for backend clients and tag-driven releases, but this edition does not claim that those workflows have run successfully on GitHub. Package names, publisher accounts, trusted publishing, and release secrets must be independently configured.

No external wire-protocol compatibility is claimed. A client package is an HTTP client with typed SQL parameters and result conversion, not a PostgreSQL driver.

Part XII — Internals and contributor guide

36. Repository tour

The root src tree contains the storage and relational foundations:

• catalog — metadata and catalog service.

• compaction — compaction structures and operations.

• error — shared engine errors.

• execution — write/batch coordination.

• lsm — integrated storage facade.

• manifest — persistent metadata edits/state.

• memtable — mutable/immutable in-memory records.

• ops — operator and catalog-facing operations.

• persist — filesystem abstraction and simulation boundary.

• relational — typed rows, table store, indexes, transactions, foreign keys.

• sstable — immutable sorted table format, readers, writers, bloom/index support.

• wal — durable append log, format, recovery, group commit, metrics.

The sql tree is intentionally explicit: parse, ast, bind, bound, auth, plan, exec, functions, cast, datetime, json, regexp, temporal, variant, copy, limits, and metrics. The test modules in sql/src/lib.rs provide a useful map of the implemented query surface.

The api tree contains configuration, auth, error mapping, state, server, SQL sessions, SQL parameters, route families, observability, security logging, and operational services. The cli tree contains client protocol code, host startup, instance commands, GUI/embedded frontend, operator commands, rendering, REPL, scripts, and startup environment handling. The instance tree owns paths, credentials, locks, manifest, port, handshake, browser, storage, volume, and relocation.

37. Making a change

A contributor should first identify the contract layer. A new SQL expression belongs in parser/AST/binder/type checking, planner/executor behavior, and corpus tests; it should not be added only to a route handler. A new route belongs in api/src/routes, route/auth/error/observability tests, client API types where applicable, and documentation. A storage change must account for WAL durability, recovery, file formats, and crash/error tests.

The repository’s source comments and architecture documents use named decisions and phase records. Preserve that traceability. Avoid adding a placeholder module for a future engine: the architecture history explicitly rejects empty directories that imply an implementation exists.

A safe contributor sequence is:

```
cargo fmt --all -- --check
cargo check --workspace --all-targets
cargo test --workspace --no-fail-fast
cargo clippy --workspace --all-targets -- -D warnings
```

Run the relevant client/frontend checks as well. The exact full gate may require a built release binary, Python optional dependencies, browser installations, or an isolated database directory. If an environment prevents a check, record it; do not convert a failure into a skip by deleting or ignoring the test.

Part XIII — Roadmap and technical limitations

38. What is present

At the covered revision, the repository provides a serious single-node stack: durable WAL operations, memtable/SSTable storage paths, catalog and relational rows, typed SQL parsing/binding/planning/execution, transactions and indexes, a protected HTTP API, local instance lifecycle, frontend console, Python and Node clients, framework helpers, bulk loading, observability, and extensive tests.

The implementation is broad, but breadth is not the same as unlimited compatibility. Each layer has explicit limits and phase boundaries. Those limits are part of the product's technical identity.

39. What is deliberately absent

The following are outside the verified current contract:

• PostgreSQL or MySQL wire protocols.

• Distributed replication, consensus, sharding, or multi-node failover.

• A Django DB-API 2.0 backend capable of making the Django ORM operate over HTTP.

• A complete API-key management lifecycle with reveal, rotate, revoke, multiple key IDs, and grace windows.

• A promise that standalone rubixdb-api.exe is the certified deployment artifact.

• A claim that every architecture document has reached runtime implementation.

• Universal cross-platform durability certification, cloud staging, or an unbounded analytical workload guarantee.

Possible future work can be inferred from architecture documents and OPEN_ITEMS.md, but this edition does not convert those records into delivery promises or dates. The correct engineering action for a missing feature is a new scoped design, implementation, test, and release decision.

Appendices

A. SQL reference map

Use these paths as the authoritative navigation map:

Concern	Source
Parser and AST	sql/src/parse, sql/src/ast
Binding and name resolution	sql/src/bind, sql/src/bound
Authorization	sql/src/auth.rs, api/src/auth.rs
Planner	sql/src/plan
Executor	sql/src/exec
Types and casts	sql/src/cast.rs, sql/src/temporal.rs, sql/src/variant.rs, src/relational/value.rs

Concern	Source
Aggregates/windows	sql/src/aggregate.rs, sql/src/exec, aggregate/window test modules
DDL/DML/constraints	phase-specific tests in sql/src and src/relational
COPY/LOAD DATA	sql/src/copy, docs/BULK_LOAD_DESIGN.md
API envelope	api/src/routes/sql.rs, api/src/sql_params.rs, frontend/src/api/types.ts

Examples should be tested with the current build. The safest first query is SELECT 1. For application values, use $1, $2, and typed client parameters.

B. CLI reference

```
rubixdb --help
rubixdb --version
rubixdb
rubixdb -c "SELECT 1"
rubixdb -f script.sql
rubixdb gui
rubixdb cli
rubixdb menu
rubixdb instance list
rubixdb instance status
rubixdb status
rubixdb check
rubixdb backup --help
rubixdb restore --help
rubixdb storage --help
rubixdb maintenance --help
```

The operator command families above are present in the CLI help or source dispatch, but subcommand options should be read from the executable's current --help output before use. Credentials must not be passed as command-line arguments.

C. Configuration reference

Core API variables are parsed in api/src/config.rs and related startup modules. Important names include:

Variable	Meaning
RUBIXDB_API_KEYS	Named API-key configuration: name:role:key,...
RUBIXDB_DATA_DIR	Standalone API data directory configuration
RUBIXDB_API_URL	Explicit client/CLI HTTP endpoint
RUBIXDB_API_KEY	Explicit client/CLI bearer key
RUBIXDB_INSTANCE_NAME	Local instance discovery name
RUBIXDB_CORS_ALLOWED_ORIGINS	Explicit comma-separated browser origins
RUBIXDB_IMPORT_DIR	Server-side import boundary where configured
RUBIXDB_BACKUP_DIR	Operator backup directory where configured

Additional variables exist for rate limits, session limits, statement deadlines, compaction, storage pressure, probes, and test/fault injection. Consult api/src/config.rs, cli/src/startup_env.rs, the instance modules, and the relevant architecture document before using one. This appendix intentionally does not invent defaults for variables not established by those sources.

D. HTTP route reference

The primary route table is assembled by api/src/routes/mod.rs. The principal families are:

```
GET    /healthz
GET    /readyz
GET    /v1/whoami
GET    /v1/status
GET    /v1/metadata
POST   /v1/sql
POST   /v1/copy
```

```
GET/PUT/DELETE /v1/kv...
GET    /v1/range
GET/POST/DELETE /v1/snapshots...
GET    /v1/catalog/*
GET    /v1/metrics*
GET    /v1/compaction/status
GET    /v1/compaction/metrics
GET    /v1/observability/sessions
GET    /v1/observability/queries
GET    /v1/observability/events
GET    /v1/observability/version
/v1/admin/*
```

The exact method/path combinations, role requirements, and response schemas are source-controlled. Use api/src/routes/mod.rs, each route module, and frontend/src/api/types.ts together; do not infer an endpoint from a frontend label alone.

E. Error and limit reference

Class	Typical meaning	Client treatment
401	Missing or invalid bearer key	Reauthenticate; do not retry blindly
403	Principal lacks required role	Change authorization/configuration
409	Write conflict or integrity conflict	Retry only a safe whole transaction; do not repeat integrity failures blindly
429	Rate/session/resource rejection	Honor retry metadata when present; session-cap errors require closing/committing transactions
5xx	Server/internal failure	Typed server error; breaker may count it
timeout	Connect/read/pool/server deadline	Typed timeout with phase
resource limit	Result, statement, body, row, or load bound	Reduce workload or configure within supported bounds

The client-specific exception names are listed in clients/python/README.md and clients/node/README.md; they should be treated as public package interfaces only at the package versions that define them.

F. Type and compatibility matrix

Interface	Current contract	Not claimed
Rust core	Workspace crate, edition 2021, version 0.1.0	Stable external Rust API policy
HTTP API	Bearer-authenticated JSON API, SQL at /v1/sql	PostgreSQL/MySQL wire protocol
Python	Python >=3.9, httpx, sync/async clients	ORM backend, distributed transport
Node	Node >=20.18.1, ESM/CJS, undici	Node 18 support for undici 7
FastAPI	Optional integration package and runnable example	Production hosting configuration for every platform
Express	Optional integration and example	Automatic deployment topology
Django	Raw SQL helper over HTTP	Django ORM over HTTP
Frontend	React/Vite console and embedded/static serving path	Key-management routes absent from server

G. Source-code map

src/lib.rs	core public modules
src/wal/	WAL format, I/O, recovery, group commit
src/memtable/	mutable/immutable memory sources
src/sstable/	RUBIC immutable sorted tables
src/manifest/	persistent metadata state
src/lsm/	integrated storage facade and recovery
src/catalog/	catalog service and metadata
src/relational/	rows, types, transactions, indexes
src/ops/	operational/catalog helpers
sql/src/lib.rs	SQL crate boundary
sql/src/parse, ast, bind, bound	SQL front end
sql/src/plan	planning and validation
sql/src/exec	physical execution
sql/src/copy	bulk loading
api/src/routes	HTTP route assembly and handlers
api/src/auth.rs	bearer auth and API-to-SQL role mapping
api/src/config.rs	environment configuration
api/src/observability	metrics, diagnostics, events
cli/src/main.rs	product CLI and help surface
cli/src/host.rs	embedded service host
instance/src/lib.rs	instance acquisition and lifecycle
frontend/src/api	typed browser HTTP client
frontend/src/integration	Data Integration UI
clients/python/rubixdb	Python clients and integrations
clients/node/src	Node client and Express adapter
examples/	runnable framework/application examples

H. Troubleshooting

The frontend displays 401. Confirm the session URL points to the active instance and the key belongs to that instance. Each instance has its own credentials. Ensure the Authorization: Bearer header is present.

The API responds 403. The key authenticated but lacks the role required by the route or SQL statement. Use an admin key for writes/admin routes and a reader key for read-only work.

The API responds 409. Determine whether the error is a lost write conflict or integrity failure. Retry the complete transaction only for a safe conflict; do not repeat a duplicate-key operation as a generic retry.

The API responds 429. It may be rate limiting or the per-principal transaction/session cap. Close abandoned transactions and honor Retry-After when supplied.

Connection refused. Run rubixdb instance list, confirm the persisted port, start target/release/rubixdb.exe, and verify that RUBIXDB_API_URL is not pointing to a stale port.

The standalone API binary exits with a configuration error. rubixdb-api.exe requires standalone variables such as RUBIXDB_DATA_DIR; use the integrated rubixdb.exe product path unless the standalone deployment is specifically being developed.

A bulk load stops after some rows. The loader commits successful batches before the first failing batch. Inspect the error class and reported progress; do not assume a global rollback.

A query is slow. Separate server execution time from client/HTTP time, inspect plan/access evidence, check row/result limits and storage state, and measure on the target machine. Do not infer a hardware ceiling from a single development run.

I. Glossary and acronyms

• **API** — application programming interface.

• **AST** — abstract syntax tree.

• **CORS** — Cross-Origin Resource Sharing.

• **CRC32C** — Castagnoli CRC used for frame integrity.

• **DDL** — data definition language.

• **DML** — data manipulation language.

• **LSM** — log-structured merge.

• **MVCC/snapshot view** — a transaction read view over a consistent version boundary; use the project's documented transaction terminology rather than assuming full PostgreSQL MVCC semantics.

• **SSTable** — immutable sorted string/table file, here the RUBIC on-disk format.

• **WAL** — write-ahead log.

• **Tombstone** — a durable deletion marker that hides older values during source merge.

• **Principal** — authenticated API identity with a role.

• **PhysicalAccess** — planner's selected access decision for a row source.

• **Session** — server-side SQL transaction state referenced by session_id.

J. Implementation inventory

The full evidence register is maintained in TECHNICAL_INVENTORY.md. It records implemented, partial, design-intent, unsupported, and unverified areas and should be updated before future editions. The book is authoritative only for the covered revision and date; changes to the source require a new review.

Closing note

RubixDB's most important engineering fact is not a single feature. It is the shape of its boundaries: a durable storage foundation, a relational layer that owns binding and authorization decisions, a service layer that exposes typed and bounded operations, and clients that preserve those contracts without pretending to be a wire-protocol database. The repository's design history is extensive because these boundaries have been negotiated and tested over time. A responsible technical reference must show both the implementation and the edges where implementation stops.